Actividad de Aprendizaje #5 De UML a código

Objetivo de la actividad:

- Que el alumno demuestre que puede **traducir un UML propio a código Java**, respetando atributos, métodos, visibilidad, constructores, static e interacción mínima mediante un Main.

Instrucciones importantes (LEE CON ATENCIÓN)

Con base en el **UML que tu equipo creó para el Proyecto Final** (sistema elegido: Estacionamiento / Consultorio / Talleres), cada integrante del equipo deberá seleccionar **2 clases** y convertirlas a código Java **exactamente como aparecen en el UML** (nombres, tipos, visibilidad y firmas). Posteriormente, creará un Main para instanciar objetos y ejecutar mensajes.

📌 **El objetivo es estructura (UML → código), no lógica completa del sistema.**

📌 No es necesario usar listas/arrays.

Desarrollo de la actividad

Parte 1. Selección desde el UML

1. De acuerdo con el UML del Proyecto Final de tu equipo.
2. Selecciona **2 clases**:
   - **1 clase entidad** (ejemplos: Vehiculo, Paciente, Taller, Inscripcion, Ticket, Cita) **Deberán escoger diferentes por cada miembro del equipo**
   - **1 clase coordinadora** (ejemplos: SistemaEstacionamiento, Consultorio, GestorTalleres)
3. **Anexa el UML,** creado en equipo, en un archivo que indique:
   - Nombre del sistema elegido
   - Nombre de las 2 clases seleccionadas

Parte 2. UML

En el mismo proyecto (o en un proyecto nuevo), crea **2 archivos .java** (uno por clase). Para **cada clase**, cumple lo siguiente:

A) Atributos

- Declara los atributos **tal como están en el UML** (mismo nombre y tipo).
- Usa **private** para atributos (equivale a - en UML).
- Si el UML marca un atributo como {static}, declárralo como static.

B) Constructores

- Implementa el constructor **default** si aparece en tu UML.
- Implementa el constructor **parametrizado** si aparece en tu UML.
- Dentro de los constructores, puedes hacer asignaciones simples (sin validaciones avanzadas).

C) Métodos

- Declara los métodos **con la misma firma del UML**:

    - mismo nombre
    - mismos parámetros (tipo y orden)
    - mismo tipo de retorno
    - visibilidad public (equivale a + en UML)
- Implementación mínima permitida:
    - si retorna int → return 0;
    - si retorna boolean → return false;
    - si retorna String → return "";
    - si retorna void → cuerpo vacío o comentario

📌 **No agregues métodos o atributos que no estén en tu UML, excepto Main.**

Parte 3. Main con objetos y mensajes

Crea Main.java y realiza:

1. Crea **mínimo 3 objetos**:
    - 2 objetos de la clase entidad
    - 1 objeto de la clase coordinadora
2. Ejecuta **mínimo 4 mensajes** (llamadas a métodos) entre objetos.
   Ejemplos de mensajes válidos:
    - sistema.registrarEntrada(vehiculo1, espacio1);
    - consultorio.agendarCita(cita1);
    - gestor.registrarInscripcion(participante1, taller1);
3. Puedes imprimir evidencias con System.out.println(...).

Parte 4. Verificación rápida (en 4 líneas)

Responde brevemente:

1. ¿Qué parte fue más directa al convertir UML a Java (atributos, constructores o métodos) y por qué?
2. ¿Qué parte te generó más duda y qué hiciste para resolverla?

Entregable

- **2 archivos .java** (las 2 clases seleccionadas)
- **Main.java** con 3 objetos y mínimo 4 mensajes
- **Respuestas de la verificación rápida** (pueden ir al final de Main como comentario)
- **Anexar UML** creado en equipo con el nombre de la actividad **Act.5-ApellidoNombre**

Criterios de evaluación

- Atributos coinciden con el UML (nombre, tipo, private)
- Constructores presentes según UML (default/parametrizado)
- Métodos coinciden con el UML (firma completa y retorno)
- Main con 3 objetos y 4 mensajes
- Verificación rápida (claridad y honestidad)

Valor: 10 pts

Fecha de entrega: Jueves **22 de enero, la actividad se cierra al finalizar la clase**
NO SE ACEPTAN ENTREGAS TARDÍAS

Lo que fue más directo al pasa fueron los métodos como de igual manera los atributos, como ya están planteados de forma correcta y es más facil pasarlos a Java.
Creo que necesito mejorar la lógica de llamada a métodos, creo que se me está complicando, siento qu en eso estoy fallando, trato de analizar los códigos antes hechos.